



Improving Counting Knapsack Problem

Prithviraj Ghosh(230794) under Prof. Sutanu Gayen



Counting Knapsack

This problem presents the situation where there are n objects each with weights $w_1, w_2, \dots, w_n$ and a constant W where we have to find the number of number of binary vectors $x \in \{0, 1\}^n$ such that $\sum_i a_i x_i \leq b$.

It is well-known that this problem is #P-hard, and considerable attention has been devoted to approximating the size of $\text{KNAP}(a, b)$. Specifically, given an error parameter ε , the goal is to compute a value p such that $(1 - \varepsilon)|\text{KNAP}(a, b)| \leq p \leq (1 + \varepsilon)|\text{KNAP}(a, b)|$, in time polynomial in both n and $1/\varepsilon$. Such a value p is referred to as a *relative-error ε -approximation*.

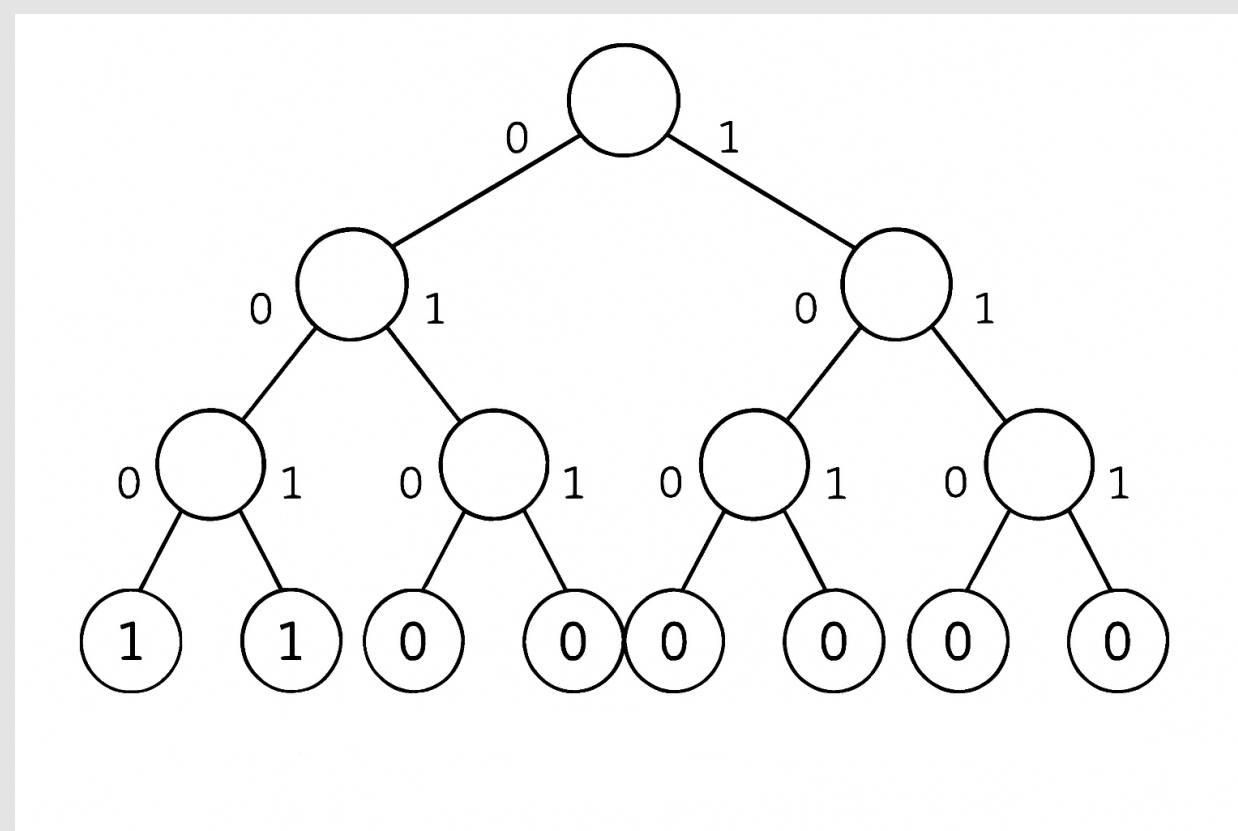
Related works

There are 2 papers which have done the task of approximating this problem very efficiently

- Polynomial-Time Approximation Schemes for Knapsack and Related Counting Problems using Branching Programs by Parikshit Gopalan, Adam Klivans, and Raghu Meka
- A Deterministic Polynomial-time Approximation Scheme for Counting Knapsack Solutions by Daniel Stefankovic, Santosh Vempala and Eric Vigoda

Branching Program

An (S, T) -branching program M is a layered multi-graph with a layer for each $0 \leq i \leq T$, and at most S vertices (states) in each layer. The first layer contains a single vertex v_0 , and each vertex in the last layer is labeled with either 0 (rejecting) or 1 (accepting). For each $0 \leq i < T$, every vertex v in layer i has two outgoing edges labeled 0 and 1, each leading to vertices in layer $i + 1$.



Dynamic Programming Approach

This approach defines a function $\tau : \{0, \dots, n\} \times \mathbb{R}_{\geq 0} \rightarrow \mathbb{R} \cup \{\pm\infty\}$ where $\tau(i, a)$ is the smallest C such that there exist at least a solutions to the knapsack problem with weights $w_1, \dots, w_i$ and capacity C . Then a recursion is established as

For any $i \in [n]$ and any $a \in \mathbb{R}_{\geq 0}$ we have

$$\tau(i, a) = \min_{\alpha \in [0, 1]} \max \left\{ \tau(i-1, \alpha a), \tau(i-1, (1-\alpha)a) + w_i \right\}$$

The solution of the problem $\max\{a : \tau(n, a) \leq C\}$ is then approximated using a geometric progression of values of a

Time Complexities

- The Branching Program approach uses a total time complexity of $O\left(\frac{n^3 \log(W) \log(n/\varepsilon)}{\varepsilon}\right)$
- The Dynamic Programming approach uses a total time complexity of $O\left(\frac{n^3 \log(n/\varepsilon)}{\varepsilon}\right)$
- The branching program uses a $\log(W)$ extra time but is useful to generate random samples
- It is used as a binary search is run over all the elements in a layer of a width- W Branching Program.

Removing the $\log(W)$ factor

We can use the function τ to directly find the elements of the branching program instead of searching for the entire layer as

Assume we have calculated v_j in the layer i along with its probability to reach an accepting state p and want to find v_{j+1} then the following equation holds exactly

$$v_{j+1} = C - \tau\left(i, \frac{2^{n-i}p}{1+\varepsilon}\right) + 1$$

which after approximation of τ achieves a bound of a constant k multiple of ε

Hamming Weight Small Space Source

A width- w small-space source is described by a (w, n) -branching program D along with a probability distribution p_v on the outgoing edges associated with each vertex $v \in D$. Samples from the source are generated by performing a random walk on D , where the transitions follow the distributions p_v , and the output is the sequence of labels on the edges traversed.

The Hamming weight of a binary string (or vector) is the number of 1s in the string.

In case of a Hamming weight Small Space Source each node s will have exactly 2 parents one with hamming weight s and the other with hamming weight $s - 1$

We want to solve the problem of Counting knapsack with this small space source which means instead of uniform probability of accepting or rejecting a weight it will follow the state in the small space source

Solution

We update the expression of τ in such a way to cater this probability as below

$$\tau(i, a, s) = \min_{\alpha, \beta \in [0, 1]} \max_{\alpha p_1 + \beta p_2 = a} \left\{ \tau(i-1, \alpha, s), \tau(i-1, \beta, s-1) + w_i \right\}$$

where p_1 & p_2 are the probabilities associated with the edge from s and $s - 1$ in layer $i - 1$ of the small space source respectively.

where $\tau(i, a, s)$ is defined as the minimum capacity with the accepting probability over all possible binary strings is at least a and present state in the small space source is s at layer i for weights $w_1, w_2, \dots, w_i$.

In order to approximate τ we use T which has 3 parameters i, j, s where all of them are natural numbers with $i \in [0, n]$, $j \in [0, k]$ where $k = \lceil n \log_{1+\delta} n \rceil$ and $s \in [0, n]$ where

$$T[i, j, s] = \min_{\substack{x, y \in [0, 1] \\ xp_1 + yp_2 = 1}} \max \left\{ T[i-1, \lfloor j + \log_{1+\delta} x \rfloor, s], T[i-1, \lfloor j + \log_{1+\delta} y \rfloor, s-1] + w_i \right\}$$

which give the following inequality

$$\tau(i, r(1+\delta)^{j-i}, s) \leq T[i, j, s] \leq \tau(i, r(1+\delta)^j, s)$$

where $r = \frac{1}{n^n}$